	AV1	AV2	AV3
1ª ch			
2ª ch			

Curso:	Disciplina:	Código/Turma:
Professor/a:		Data:
Aluno/a:		Matrícula:

INSTRUÇÕES PARA O DESENVOLVIMENTO DO TRABALHO

- Trabalho é em Equipe (até 4 pessoas).
- O Trabalho vale 30% da nota da disciplina e o trabalho será apresentado pela equipe. Será marcado um dia para a equipe mostrar a implementação e a execução da aplicação.

Implementar um índice hash estático

Levantamento de Requisitos (Histórias de Usuário)

1) Visão Geral do Produto

O sistema será uma aplicação que simula o funcionamento de um índice hash estático sobre uma tabela de dados armazenada em páginas na memória. O sistema deverá permitir:

- Carregar um arquivo texto com palavras únicas;
- Dividir o conjunto de registros em páginas;
- Construir um índice hash estático com buckets;
- Pesquisar por uma chave usando o índice;
- Comparar a busca por índice com um table scan;
- Apresentar estatísticas de colisões, overflow e custo estimado (leitura de páginas).
- Arquivos de dados: contém os dados que serão usados para popular uma ou mais tabelas. Para este trabalho será usado um arquivo com 466 mil palavras do idioma Inglês, disponível em: https://github.com/dwyl/english-words. Este arquivo txt tem somente uma palavra por linha. Esta única palavra é única no arquivo, podendo ser considerada chave.
- Interface gráfica obrigatória ilustrando as estruturas de dados e o funcionamento de um índice hash estático
- Deve ser calculada e mostrada a taxa de colisões (%).
- Deve ser calculada e mostrada a taxa de overflows (%).
- Deve ser calculado e mostrado uma estimativa de custo (quantidade de acessos a disco – leitura de páginas), quando uma chave de busca é entrada.
- Deve ser calculado e mostrado a estimativa de custo (quantidade de páginas lidas) para o table scan.
- Deve ser mostrada a diferença de tempo entre a busca usando o índice e o table scan.
- A implementação do índice deve levar em consideração as colisões, ou seja, deve ser implementado um algoritmo de resolução de colisões.
- A implementação do índice deve levar em consideração o transbordamento dos buckets (bucket overflow), ou seja, deve ser implementado um algoritmo de resolução de overflow.

2) Glossário

- Tupla / Registro: neste projeto, uma palavra (string) lida do arquivo.
- Chave de busca: a palavra (única no arquivo) usada para busca.
- Página: unidade de armazenamento lógico que agrupa registros.
- Bucket: estrutura do índice hash que armazena pares (chave → endereço da página).
- FR: capacidade do bucket (quantas chaves ele suporta).
- NB: número total de buckets.

- Colisão: Quando mais de uma chave diferente leva para um mesmo bucket (ou ultrapassa o tamanho (FR) definido para o bucket).
- Overflow: bucket excede FR e precisa de estratégia de transbordamento.
- Table Scan: leitura sequencial das páginas até achar a chave.

3) Atores

- Aluno/Usuário final: usa a interface, carrega dados, constrói índice e executa buscas.
- Sistema: processa dados, constrói índice e calcula métricas.

4) Requisitos em Histórias de Usuário

EPIC 1 — Carga e Organização dos Dados

HU01 — Carregar arquivo de palavras

Como usuário
Quero carregar um arquivo TXT contendo palavras únicas
Para popular as páginas em memória (dados da tabela) a ser indexada.

Regras de Negócio

RN01: O arquivo deve conter uma palavra por linha.
RN02: Cada palavra do arquivo deve ser considerada uma chave única.
RN03: O sistema deve suportar o arquivo com aproximadamente 466 mil palavras.

Critérios de Aceitação

CA01: O sistema permite selecionar um arquivo .txt e carrega os registros.
CA02: O sistema informa o total de palavras carregadas.
CA03: O sistema trata e informa erro se o arquivo estiver vazio ou ilegível.

HU02 — Definir tamanho de página

Como usuário
Quero informar o tamanho da página (registros por página)
Para controlar como os registros serão divididos em páginas de banco de dados.

Regras de Negócio

RN04: O tamanho da página deve ser uma entrada do usuário (digitado na interface).
RN05: O tamanho da página deve ser maior que zero.

Critérios de Aceitação

CA04: Existe um campo na interface para digitar o tamanho da página.
CA05: Se o valor for inválido (zero, negativo ou vazio), o sistema impede a continuação.

HU03 — Dividir registros em páginas

Como usuário
Quero que o sistema divida automaticamente os registros em páginas
Para simular armazenamento físico em disco.

Regras de Negócio

RN06: A divisão em páginas depende diretamente do tamanho definido pelo usuário.
RN07: O número de páginas é calculado automaticamente → Quantidade de registros dividido pela quantidade de registros por página definido na HU02.

Critérios de Aceitação

CA06: Após carregar o arquivo, o sistema exibe a quantidade total de páginas calculada.
CA07: O sistema exibe na interface a primeira e a última página, com:

1. Número da página
2. Primeiros 5 Registros contidos nela

EPIC 2 — Construção do Índice Hash Estático

HU04 — Criar buckets do índice

Como usuário
Quero que o sistema crie automaticamente os buckets do índice
Para permitir a construção do índice hash estático.

Regras de Negócio

RN08: O número de buckets **NB** deve obedecer:

- NB > NR / FR, onde NR é o número de registros e FR é o tamanho do bucket.

RN09: FR (capacidade do bucket) deve ser definido pela equipe.

Critérios de Aceitação

CA08: O sistema calcula e exibe **NB**.
CA09: O sistema cria NB buckets com capacidade FR.
CA10: O sistema impede NB <= NR/FR.

HU05 — Implementar função hash configurável

Como usuário
Quero que o índice utilize uma função hash definida pela equipe
Para mapear chaves de busca em buckets.

Regras de Negócio

RN10: A função hash deve mapear uma chave em um endereço de bucket.
RN11: A função hash deve ser escolhida/projetada pela equipe.

Critérios de Aceitação

CA11: Dada uma chave, o sistema retorna sempre o mesmo bucket.
CA12: A função hash sempre retorna um bucket dentro do intervalo válido [0..NB-1].

HU06 — Construir o índice percorrendo as páginas

Como usuário
Quero que o sistema construa o índice percorrendo página por página
Para simular o custo real de leitura de dados.

Regras de Negócio

RN12: A construção do índice percorre páginas e registros.
RN13: Para cada tupla (registro – neste exemplo temos apenas a chave que foi lida do arquivo):

1. aplica-se a função hash
2. armazena-se no bucket:
    a. chave de busca
    b. endereço da página onde o registro está (página de banco de dados)

Critérios de Aceitação

CA13: Ao final, o índice contém todos os registros do arquivo.
CA14: O sistema exibe o tempo de construção do índice.

EPIC 3 — Tratamento de Colisões e Overflow

HU07 — Resolver colisões

Como usuário
Quero que o índice trate colisões
Para permitir que várias chaves que caem no mesmo bucket sejam armazenadas corretamente.

Regras de Negócio

RN14: A implementação deve considerar colisões. Serão consideradas colisões somente aquelas que ultrapassem o tamanho do bucket. Só irá contabilizar colisões quando o bucket estiver cheio.
RN15: A equipe deve implementar um algoritmo de resolução de colisões.

Critérios de Aceitação

CA15: O sistema insere registros mesmo quando múltiplas chaves geram o mesmo bucket.
CA16: O sistema contabiliza colisões que excedem o tamanho definido para o Bucket..

HU08 — Resolver overflow de buckets

Como usuário
Quero que o índice trate overflow de buckets
Para garantir que registros adicionais sejam armazenados quando um bucket exceder FR.

Regras de Negócio

RN16: A implementação deve considerar bucket overflow.
RN17: Deve existir algoritmo de resolução de overflow.

Critérios de Aceitação

CA17: Quando FR for excedido, o sistema usa a estratégia implementada (bucket overflow).
CA18: O sistema contabiliza quantos buckets entraram em overflow.

EPIC 4 — Pesquisa por Índice

HU09 — Buscar uma chave usando o índice

Como usuário
Quero digitar uma chave de busca e executar a busca via índice
Para localizar rapidamente o registro e sua página.

Regras de Negócio

RN18: Deve existir um campo para digitar a chave de busca.
RN19: A busca deve:

1. aplicar a função hash na chave
2. localizar o bucket
3. recuperar o endereço da página
4. carregar a página e localizar a tupla

Critérios de Aceitação

CA19: Ao buscar, o sistema mostra:

1. se a chave foi encontrada
2. em qual página está
3. custo estimado em leituras de página

CA20: Se a chave não existir, o sistema informa “não encontrada”.

EPIC 5 — Table Scan e Comparação

HU10 — Executar table scan até encontrar a chave

Como usuário
Quero executar um table scan após informar uma chave
Para comparar a busca indexada com busca sequencial.

Regras de Negócio

RN20: Deve existir um botão de table scan habilitado após digitar uma chave.
RN21: O table scan deve listar os registros lidos até encontrar a chave, lendo página por página até encontrar o valor chave.

Critérios de Aceitação

CA21: O sistema exibe os registros lidos durante o scan.
CA22: O sistema informa:

1. número da página onde encontrou
2. custo (quantidade de páginas lidas)

HU11 — Comparar tempo e custo entre índice e scan

Como usuário
Quero ver a diferença de tempo e custo entre índice e scan
Para entender o ganho real de usar índice.

Regras de Negócio

RN22: Deve ser mostrada a diferença de tempo entre busca com índice e table scan.
RN23: Deve ser estimado custo em acessos a disco/leitura de páginas.

Critérios de Aceitação

CA23: O sistema exibe tempo de execução de:

1. busca com índice
2. table scan

CA24: O sistema exibe custo estimado (páginas lidas) de ambos e uma diferença percentual das buscas.

EPIC 6 — Estatísticas e Métricas

HU12 — Calcular taxa de colisões

Como usuário
Quero visualizar a taxa de colisões
Para avaliar a qualidade da função hash e do Numero de Buckets (NB).

Regras de Negócio

RN24: Deve ser calculada e exibida a taxa de colisões (%).

Critérios de Aceitação

CA25: A interface mostra o percentual de colisões após construir o índice.

HU13 — Calcular taxa de overflow

Como usuário
Quero visualizar a taxa de overflow
Para avaliar se FR e NB foram dimensionados corretamente.

Regras de Negócio

RN25: Deve ser calculada e exibida a taxa de overflow (%).

Critérios de Aceitação

CA26: A interface mostra o percentual de overflow após construir o índice.

EPIC 7 — Interface Gráfica e Visualização

HU14 — Visualizar estruturas e funcionamento

Como usuário
Quero uma interface gráfica que mostre as estruturas e o funcionamento do índice
Para entender visualmente como páginas, buckets e buscas funcionam.

Regras de Negócio

RN26: Interface gráfica é obrigatória.
RN27: A interface deve ilustrar:

1. páginas
2. buckets
3. processo de busca
4. localização do registro

Critérios de Aceitação

CA27: O usuário consegue ver a primeira e última página.
CA28: O usuário consegue ver buckets e seus conteúdos.

CA29: Durante a busca, o bucket e a página acessados são destacados.

5) Requisitos Não Funcionais (RNF)

RNF01: O sistema deve suportar pelo menos 466.000 registros sem travar.
RNF02: O tempo de construção do índice deve ser exibido.
RNF03: O sistema pode ser construído em qualquer linguagem (Java, Python, C#, JS, etc.).
RNF04: A interface pode ser desktop ou web, mas deve ser visual. Não será aceito programas rodando no terminal com uso janelas popup.
RNF05: O sistema deve ser determinístico: mesma entrada (chave) → mesmo índice.

6) Funcionamento em passos:

Carga do programa:

a) O arquivo de dados é carregado em memória.
b) As linhas da tabela devem ser divididas em páginas, de acordo com o tamanho das páginas.
c) Deve ser mostrada na interface a primeira e a última página carregada, mostrando o nome/numero da página e os registros que estão em cada uma das páginas.
d) NB buckets de tamanho FR são criados (NB > NR/FR, onde NR representa o número de tuplas e FR representa o número de tuplas por Bucket).
e) Para a construção do índice deve-se percorrer página por página, pegando cada uma das chaves de busca que foram carregadas nas páginas.
f) A função hash é aplicada à chave de busca de cada tupla; a chave de busca e o endereço da página onde a tupla foi armazenada são adicionadas ao bucket cujo endereço foi calculado pela função hash.

Uso do programa:

g) Uma chave de busca é fornecida na interface.
h) A função hash deve ser aplicada na chave de busca fornecida encontrando a página de dados a ser lida
i) A página deve ser lida e será informado na interface gráfica se a chave foi encontrada, o número da página e o custo da leitura.
j) O botão table scan deve ser acionado e o resultado da leitura das páginas deve ser mostrado na interface gráfica, além do número da página onde a chave foi encontrada e o custo da leitura do table scan.
k) Deve ser mostrada a diferença de tempo entre a busca usando a chave e o table Scan.

Critério	Notas
1. Interface gráfica (funcional)	1,0
2. Carga de Dados nas páginas (funcional e código fonte)	1,5
3. Entrada para Tamanho da página (funcional e código fonte)	1,0
4. Cálculo da quantidade de páginas (funcional e código fonte)	1,0
5. Construção e uso da função hash (funcional e código fonte)	1,0
6. Cálculo da quantidade de buckets (funcional e código fonte)	0,5
7. Funcionamento com pesquisa com o uso do índice (funcional e código fonte)	2,0
8. Deve ser calculada e mostrada a taxa de colisões. (funcional)	0,5
9. Deve ser calculada e mostrada a taxa de overflows. (funcional)	0,5
10. Execução do Table Scan (funcional)	0,5
11. Deve ser calculado e mostrado uma estimativa de custo (acessos a disco) e o comparativo de tempo entre a busca com índice e o table scan (funcional)	0,5

*funcional – Será analisada apenas a funcionalidade
** funcional e código fonte - Será analisada a funcionalidade e será necessário explicar o código fonte.